

INTERVIEW PREPARATION

Spring Boot Interview Notes

Senior software engineer level — questions, full answers, code, trade-offs and the follow-ups interviewers ask next.

68 questions across 12 sections

Core container · auto-configuration internals · JPA & transactions · REST design · security · testing · microservices · performance · observability

Written against **Spring Boot 4.1** · September 2026

Contents

How to Use These Notes	5
Currency	5
Suggested revision order	5
1. Core Spring & the IoC Container	6
Q1. Why is constructor injection preferred over field injection?	6
Q2. Walk me through the full bean lifecycle.	6
Q3. What does <code>proxyBeanMethods = true</code> on <code>@Configuration</code> actually do?	7
Q4. How does Spring resolve circular dependencies, and when can't it?	8
Q5. How do you inject a prototype bean into a singleton?	9
Q6. <code>@Component</code> scanning vs <code>@Bean</code> methods — when do you use which?	9
Q7. Why does self-invocation break <code>@Transactional</code> and <code>@Async</code> ?	9
Q8. Multiple beans of the same type — how does Spring pick one?	10
Q9. What is <code>SmartLifecycle</code> for, and how does it differ from <code>@PostConstruct</code> ?	10
2. Auto-Configuration Internals & Custom Starters	12
Q10. Explain how auto-configuration works, end to end.	12
Q11. Walk through the <code>@Conditional</code> family.	13
Q12. How would you write a custom starter?	13
Q13. An auto-configuration isn't applying. How do you debug it?	14
Q14. Why <code>@AutoConfigureAfter</code> instead of <code>@Order</code> ?	15
Q15. What is a <code>FailureAnalyzer</code> ?	15
Q16. How does Spring Boot's modularization in 4.0 change the auto-configuration story?	15
3. Configuration, Profiles & Externalized Config	17
Q17. What is the property resolution order in Spring Boot?	17
Q18. <code>@Value</code> vs <code>@ConfigurationProperties</code> ?	17
Q19. How do profiles work, and how should they be used in production?	18
Q20. How do you handle secrets?	19
Q21. What does <code>spring.config.import</code> give you that <code>@PropertySource</code> doesn't?	19
Q22. How do you change configuration without a restart?	19
4. REST API Design	21
Q23. How do you version a REST API?	21
Q24. How should errors be returned?	22
Q25. How do you make an endpoint idempotent?	22
Q26. <code>RestTemplate</code> , <code>WebClient</code> , or HTTP interfaces?	23
Q27. How do you paginate correctly?	24
Q28. Where does validation belong?	24
5. Data Access, JPA & Transactions	26

Q29. What is the N+1 select problem and how do you fix it?	26
Q30. What exactly does <code>@Transactional</code> do, and what are its pitfalls?	27
Q31. Explain propagation levels.	28
Q32. Optimistic vs pessimistic locking?	28
Q33. How do you size a connection pool?	29
Q34. <code>LazyInitializationException</code> — cause and correct fix?	29
Q35. How do you manage schema migrations?	30
6. Spring Security & JWT	31
Q36. Explain the Spring Security filter chain.	31
Q37. Authentication vs authorization, and where does each happen?	31
Q38. How do you validate a JWT, and what do people get wrong?	32
Q39. How does CSRF work, and when can you disable it?	33
Q40. How do you store passwords?	33
Q41. What are the main security issues you would look for in a Spring Boot service?	34
7. Testing Strategy	35
Q42. What are test slices and why do they matter?	35
Q43. Testcontainers vs H2 for repository tests?	36
Q44. How do you test a <code>@Transactional</code> service method?	36
Q45. How do you test asynchronous or scheduled code?	37
Q46. What is your view on mocking?	37
Q47. What changed for testing in Spring Boot 4?	37
8. Microservices & Resilience Patterns	39
Q48. How do you decide service boundaries?	39
Q49. How do you make service-to-service calls resilient?	39
Q50. How do you handle distributed transactions?	40
Q51. Sync or async between services?	41
Q52. How do you do service discovery and config in Kubernetes?	41
Q53. How do you keep data consistent across services that need each other's data?	41
9. Performance, Concurrency & Scaling	43
Q54. What are virtual threads and when should you use them?	43
Q55. WebFlux or MVC?	43
Q56. How do you improve Spring Boot startup time?	44
Q57. How do you add caching, and what are the traps?	45
Q58. A production endpoint got slow. Walk me through your diagnosis.	45
Q59. How do you scale a Spring Boot service?	46
10. Observability & Production Readiness	47
Q60. What are the three pillars, and what does Spring Boot give you for each?	47
Q61. What makes a good health check?	47
Q62. How does distributed tracing work, and what is context propagation?	48

Q63. What is metric cardinality and why does it matter?	49
Q64. How would you structure logging for a distributed system?	49
Q65. What is your production readiness checklist for a new service?	49
11. Spring Boot 4 — What Changed and Why	51
Q66. What is new in Spring Boot 4 / Spring Framework 7?	51
Q67. What actually breaks when you upgrade from Boot 3 to Boot 4?	51
Q68. Should a team upgrade to Boot 4 now?	52
12. Rapid-Fire Round	53
Closing question to ask them	54

How to Use These Notes

These are interview notes for a **senior** Spring Boot role, which changes what the questions are really asking. At mid-level, "what does `@Transactional` do?" is a knowledge question. At senior level the same words mean: *do you know where it silently fails to apply, what it costs in connection-pool terms, and how you would explain that to someone on your team?* Almost every answer here therefore has the same shape — the mechanism, the trade-off, and the failure mode.

Three things worth knowing about how senior rounds are scored.

Trade-offs beat recall. An answer that names a cost is worth more than an answer that is merely correct. "We used Testcontainers because H2's dialect differences made green tests meaningless, accepting slower CI and a Docker dependency" says more than any definition of Testcontainers.

Depth is probed, not breadth. Interviewers pick one answer and push on it three times. Answers here include a **Follow-up** line where the next question is predictable, so you can prepare the second and third layer rather than only the first.

Saying "I don't know" is a positive signal when followed by how you would find out. Bluffing is the most reliably fatal move in a senior interview, because the interviewer usually knows the answer and is watching what you do at the edge of your knowledge.

Currency

Written against **Spring Boot 4.1** (current as of September 2026; 4.1.0 released June 2026). Spring Boot 3.5 left open-source support on 30 June 2026. Section 11 covers what changed in 4.x and what breaks on upgrade — worth reading even if your current job is on 3.x, because "are you keeping up?" is a question interviewers ask precisely because it is cheap to ask and revealing to answer.

Code samples are complete enough to discuss but trimmed of imports and boilerplate.

Suggested revision order

If you have a week, read sections 1, 2 and 5 first — core container, auto-configuration internals, and transactions are where the deepest probing happens and where most candidates run out of material. Sections 8 through 10 (microservices, performance, observability) are where senior candidates are separated from mid-level ones, because the questions stop being about Spring and start being about systems. Section 12 is a night-before skim.

1. Core Spring & the IoC Container

The interviewer's goal in this round is to find out whether you understand the container as a program you can reason about, or whether Spring is magic that happens to work.

Q1. Why is constructor injection preferred over field injection?

Four reasons, roughly in order of how much interviewers care:

Immutability and a valid object at all times. Constructor injection lets every collaborator be `final`. The object cannot exist in a half-built state, so there is no window where a dependency is `null`.

Honest API. A constructor with nine parameters is uncomfortable to write, and that discomfort is the point — it is the class telling you it has too many responsibilities. Field injection hides the same nine dependencies behind annotations and removes the design pressure.

Testability without the container. A constructor-injected class is a plain Java object you can instantiate in a unit test. Field injection forces you into reflection or a Spring context.

Circular dependencies fail fast. With constructor injection a cycle is a startup failure instead of a subtle runtime problem.

```
@Service
public class PaymentService {
    private final PaymentGateway gateway;
    private final PaymentRepository repository;

    // No @Autowired needed – single constructor is auto-detected since Spring 4.3
    public PaymentService(PaymentGateway gateway, PaymentRepository repository) {
        this.gateway = gateway;
        this.repository = repository;
    }
}
```

Trade-off worth naming. Field injection is genuinely more convenient in test classes, and `@Autowired` on fields is still idiomatic there. Setter injection remains the right choice for truly optional dependencies and for reconfigurable beans.

Follow-up you should expect: "So how would you handle a genuine circular dependency?" — see Q4.

Q2. Walk me through the full bean lifecycle.

The answer that separates senior from mid-level is knowing that there are **two distinct phases**: the container first builds a model of your beans, then instantiates them.

Phase 1 — definition. Bean definitions are loaded from component scanning, `@Bean` methods, and XML or programmatic registration. Nothing is instantiated yet. `BeanFactoryPostProcessor` runs here and can *modify definitions* — this is how `PropertySourcesPlaceholderConfigurer` resolves `${...}` placeholders before any bean is built. `BeanDefinitionRegistryPostProcessor` is a sub-interface that can also *add* definitions, which is how Mapper/repository scanning works.

Phase 2 — instantiation, for each non-lazy singleton:

1. Constructor invoked (dependencies resolved recursively).
2. Dependency injection of fields and setters.
3. `*Aware` callbacks — `BeanNameAware`, `BeanFactoryAware`, `ApplicationContextAware`.
4. `BeanPostProcessor.postProcessBeforeInitialization`.
5. `@PostConstruct`, then `InitializingBean.afterPropertiesSet()`, then a custom `initMethod`.
6. `BeanPostProcessor.postProcessAfterInitialization` — **this is where AOP proxies are created**, which is why the proxy wraps a fully initialized target.
7. Bean is ready and cached in the singleton registry.

Shutdown: `@PreDestroy`, then `DisposableBean.destroy()`, then the custom `destroyMethod`. Note that prototype beans never get destruction callbacks — the container hands them over and forgets them.

```
@Component
public class ConnectionWarmer implements BeanPostProcessor {
    @Override
    public Object postProcessAfterInitialization(Object bean, String name) {
        if (bean instanceof DataSource ds) {
            // runs after the DataSource is fully initialised, before anyone uses it
            try (var c = ds.getConnection()) { c.isValid(1); } catch (SQLException e) { }
        }
        return bean;
    }
}
```

Follow-up: "At which step does `@Transactional` take effect?" Step 6 — the proxy is created by `AbstractAutoProxyCreator`, a `BeanPostProcessor`.

Q3. What does `proxyBeanMethods = true` on `@Configuration` actually do?

A `@Configuration` class is itself CGLIB-subclassed by default. That subclass intercepts calls to `@Bean` methods so that calling one *inside* the configuration class returns the cached singleton instead of executing the method again.

```
@Configuration // full mode: proxyBeanMethods = true (default)
public class AppConfig {
    @Bean public Cache cache() { return new Cache(); }
    @Bean public Service service() {
        return new Service(cache()); // intercepted -> returns the SAME Cache singleton
    }
}
```

With `proxyBeanMethods = false` ("lite mode") there is no subclass, so `cache()` is a plain Java call and `service()` receives a **second, unmanaged** `Cache` instance. Lite mode is faster to start and is what Spring Boot's own auto-configuration classes use — they are written to never inter-call `@Bean` methods, taking dependencies as method parameters instead:

```
@Configuration(proxyBeanMethods = false)
public class AppConfig {
    @Bean public Cache cache() { return new Cache(); }
    @Bean public Service service(Cache cache) { return new Service(cache); } // correct
}
```

Why it matters: it is a real, silent bug source. Two instances of a cache or a connection pool where you expected one.

Q4. How does Spring resolve circular dependencies, and when can't it?

Spring's singleton factory keeps **three maps**: fully initialized singletons, early singleton *factories*, and early singleton *references*. When bean A is being created, a reference to the raw, not-yet-populated A is exposed through the early-reference map. If B then asks for A, it gets that early reference, finishes building, and A completes.

That trick only works if A can be **constructed** before B exists. So:

- **Setter/field injection cycle** — resolvable, because construction and injection are separate steps.
- **Constructor injection cycle** — unresolvable. Neither object can be constructed first. `BeanCurrentlyInCreationException` at startup.

Since Boot 2.6, circular references are **disabled by default**; `spring.main.allow-circular-references=true` re-enables them.

The right answer to "how do you fix it" is not to flip that flag. A cycle almost always means a missing abstraction: extract the shared behaviour into a third collaborator that both depend on, or introduce an event so the dependency becomes one-directional. `@Lazy` on one side works — it injects a proxy that resolves on first use — but it is a bandage over a design smell.

Follow-up: "Why does `@Lazy` break the cycle?" Because the proxy is constructible without the target; the real lookup is deferred to first method call.

Q5. How do you inject a prototype bean into a singleton?

Naively, you can't — the singleton is injected once, so it holds one prototype instance forever. Four options, best first:

```
// 1. ObjectProvider - explicit, testable, no proxying magic
@Service
public class ReportService {
    private final ObjectProvider<ReportJob> jobs;
    ReportService(ObjectProvider<ReportJob> jobs) { this.jobs = jobs; }
    public void run() { ReportJob job = jobs.getObject(); job.execute(); }
}

// 2. Scoped proxy on the prototype/request bean
@Bean @Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)
public ReportJob reportJob() { return new ReportJob(); }

// 3. @Lookup - container overrides the abstract method
@Component
public abstract class ReportService {
    @Lookup protected abstract ReportJob createJob();
}
```

Option 4 is `ApplicationContextAware` plus `getBean()`, which works but couples your code to the container — mention it, then say you would avoid it.

The same question applies to request-scoped beans in a singleton, where the answer is the scoped proxy: the proxy looks up the correct instance from the current request's scope on each call.

Q6. `@Component` scanning vs `@Bean` methods — when do you use which?

`@Component` (and its stereotypes) is for **your own classes**: the class is yours to annotate and the wiring is uninteresting. `@Bean` is for **types you do not own** — a third-party client, a `DataSource`, an `ObjectMapper` — and for cases where construction needs logic, conditions, or several variants of the same type.

Senior nuance: heavy component scanning across broad packages has a measurable startup cost, and scanning `basePackages = "com"` is a classic accident. Boot's `@SpringBootApplication` scans from the annotated class's package downward — which is why the main class belongs in the **root** package.

Q7. Why does self-invocation break `@Transactional` and `@Async`?

Because both are implemented with **proxies**, not bytecode weaving. Callers get a proxy; the proxy applies the advice and then delegates to the target. A call from one method of the target to another goes through `this`, never touching the proxy, so no advice runs.

```

@Service
public class OrderService {
    public void placeOrder(Order o) {
        validate(o);
        save(o); // <-- self-invocation: @Transactional here does NOTHING
    }
    @Transactional
    public void save(Order o) { /* ... */ }
}

```

Fixes, in order of preference: **move save into a separate bean** (usually the right design); inject a self-reference (`@Lazy OrderService self`); use `TransactionTemplate` programmatically; or enable AspectJ load-time weaving (`spring.aop.auto + @EnableTransactionManagement(mode = ASPECTJ)`), which most teams should avoid for the operational complexity.

Related detail: Spring uses **JDK dynamic proxies** when the bean implements an interface and **CGLIB** otherwise — though Spring Boot forces CGLIB by default (`spring.aop.proxy-target-class=true`). CGLIB cannot proxy `final` classes or `final` methods, which is why a `final` method silently loses its `@Transactional`.

Q8. Multiple beans of the same type — how does Spring pick one?

Resolution order: **by type** → if several, narrow by `@Qualifier` → then `@Primary` → then match the **field/parameter name** against bean names → otherwise `NoUniqueBeanDefinitionException`.

Generics count as part of the type, which is genuinely useful:

```

@Bean public Validator<Order> orderValidator() { ... }
@Bean public Validator<User> userValidator() { ... }

@Service
class OrderService {
    OrderService(Validator<Order> validator) { } // unambiguous — generics are matched
}

```

For a *collection* of implementations, inject `List<T>` (ordered by `@Order/Ordered`) or `Map<String, T>` (keyed by bean name) — the standard way to implement a strategy registry.

Q9. What is `SmartLifecycle` for, and how does it differ from `@PostConstruct`?

`@PostConstruct` runs during **bean** initialization — other beans may not exist yet, and the web server is definitely not listening. `SmartLifecycle` runs after the **context** is refreshed, in a controlled phase order, and gives you a matching `stop(Runnable)` for graceful shutdown.

Use it for anything that starts consuming or accepting traffic: a Kafka consumer, a scheduler, a background poller. Start in a high phase so it comes up after infrastructure; stop early so you drain before the datasource closes.

```
@Component
public class ConsumerLifecycle implements SmartLifecycle {
    private volatile boolean running;
    @Override public void start() { consumer.subscribe(); running = true; }
    @Override public void stop() { consumer.close(); running = false; }
    @Override public boolean isRunning() { return running; }
    @Override public int getPhase() { return Integer.MAX_VALUE - 100; } // late start
}
```

Pair it with `server.shutdown=graceful` and `spring.lifecycle.timeout-per-shutdown-phase=30s` so in-flight requests finish before the JVM exits — a question that comes up constantly for anyone running on Kubernetes.

2. Auto-Configuration Internals & Custom Starters

This is the single most reliable place to demonstrate that you know Spring Boot rather than just use it. Most candidates can say "it configures things automatically". Very few can trace the mechanism.

Q10. Explain how auto-configuration works, end to end.

`@SpringBootApplication` is a composite of `@SpringBootConfiguration`, `@ComponentScan`, and `@EnableAutoConfiguration`. The last one imports `AutoConfigurationImportSelector`, and that class does the work:

1. **Discovery.** It reads every `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` file on the classpath — a plain newline-delimited list of class names. (Before Boot 2.7 this was the `EnableAutoConfiguration` key in `spring.factories`; that form was removed in Boot 3.)
2. **Filtering.** Exclusions are removed, then `AutoConfigurationImportFilter`s run. These use the pre-computed `spring-autoconfigure-metadata.properties` index to reject candidates on cheap classpath checks *before* the classes are even loaded — the main startup optimisation.
3. **Ordering.** Survivors are sorted by `@AutoConfigureOrder`, `@AutoConfigureBefore`, and `@AutoConfigureAfter`.
4. **Conditional evaluation.** Each remaining class is processed as a `@Configuration` class, and its `@Conditional` annotations decide whether its beans are registered.

The crucial detail is `@ConditionalOnMissingBean`: auto-configuration always backs off if you have defined the bean yourself. That is the whole reason Boot feels like it "gets out of your way" — your configuration is processed as a user configuration class, and auto-configurations are ordered to run **last**.

```
@AutoConfiguration(after = DataSourceAutoConfiguration.class)
@ConditionalOnClass(JdbcTemplate.class)
@ConditionalOnSingleCandidate(DataSource.class)
public class JdbcTemplateAutoConfiguration {
    @Bean
    @ConditionalOnMissingBean(JdbcOperations.class) // you win if you declared one
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
        return new JdbcTemplate(dataSource);
    }
}
```

Follow-up: "Why must user configuration be processed before auto-configuration?" Because `@ConditionalOnMissingBean` is evaluated against the bean definitions registered so far. Order is the mechanism, not a nicety.

Q11. Walk through the `@Conditional` family.

Annotation	Fires when	Notes
<code>@ConditionalOnClass</code> / <code>@ConditionalOnMissingClass</code>	Class present/absent on classpath	Evaluated via ASM without loading the class
<code>@ConditionalOnBean</code> / <code>@ConditionalOnMissingBean</code>	Bean definition present/absent	Order-sensitive
<code>@ConditionalOnSingleCandidate</code>	Exactly one candidate, or one <code>@Primary</code>	Used for <code>DataSource</code>
<code>@ConditionalOnProperty</code>	Property matches	<code>matchIfMissing</code> controls the default
<code>@ConditionalOnWebApplication</code>	Servlet / reactive / any	<code>type = Type.SERVLET</code>
<code>@ConditionalOnResource</code>	Resource on classpath	e.g. a config file
<code>@ConditionalOnExpression</code>	SpEL evaluates true	Escape hatch; harder to read
<code>@ConditionalOnThreading</code>	Platform vs virtual threads	Added for virtual-thread support

The trap interviewers probe: `@ConditionalOnBean` **inside your own `@Configuration` class is dangerous**. Bean definitions are registered in an undefined order among user configuration classes, so the condition may evaluate before the bean it is looking for is registered. It is only reliable in auto-configuration, where ordering is explicit. If you find yourself reaching for it in application code, that is usually a sign the logic belongs in an auto-configuration or behind a property switch instead.

Q12. How would you write a custom starter?

Convention: **two modules**. An `autoconfigure` module holding the logic, and a starter module that is a pom with dependencies only. Naming: `acme-spring-boot-starter` — the `spring-boot-starter-*` prefix is reserved for official starters.

```
// 1. Type-safe properties
@ConfigurationProperties(prefix = "acme.audit")
public record AuditProperties(
    @DefaultValue("true") boolean enabled,
    @DefaultValue("30d") Duration retention,
    String endpoint) { }

// 2. The auto-configuration
@Configuration
@ConditionalOnClass(AuditClient.class)
@ConditionalOnProperty(prefix = "acme.audit", name = "enabled", matchIfMissing = true)
@EnableConfigurationProperties(AuditProperties.class)
public class AuditAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public AuditClient auditClient(AuditProperties props) {
        return new AuditClient(props.endpoint(), props.retention());
    }
}
```

```
# 3. src/main/resources/META-INF/spring/
# org.springframework.boot.autoconfigure.AutoConfiguration.imports
com.acme.audit.AuditAutoConfiguration
```

Points that earn credit: always guard beans with `@ConditionalOnMissingBean` so consumers can override; add `spring-boot-configuration-processor` as an optional dependency so IDEs get autocomplete for your properties; ship sensible defaults; never use `@ComponentScan` in a starter (it would scan the *consumer's* packages unpredictably); and add `@AutoConfiguration(after = ...)` rather than relying on luck.

Q13. An auto-configuration isn't applying. How do you debug it?

Start with the **condition evaluation report** — it is the purpose-built tool and knowing it signals real experience:

```
java -jar app.jar --debug
# or
management.endpoints.web.exposure.include=conditions # then GET /actuator/conditions
```

It prints three groups: *positive matches* (applied, and why), *negative matches* (skipped, and which condition failed), and *exclusions*. Nine times out of ten the negative-match reason is the whole answer — a missing class, a property not set, or a bean you unknowingly already defined.

Then the usual suspects: the main class is not in a parent package of your components; a dependency is missing so `@ConditionalOnClass` fails; you defined a competing bean and the auto-config backed off; or the auto-configuration jar is shaded/repackaged in a way that lost the `.imports` file.

To force the issue: `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)` or `spring.autoconfigure.exclude=...`.

Q14. Why `@AutoConfigureAfter` instead of `@Order` ?

They operate on different things. `@Order` sorts *beans* of the same type in a collection, and orders AOP advice. `@AutoConfigureBefore/After/Order` sort the *processing of configuration classes*, which determines the order bean definitions are registered — and therefore how `@ConditionalOnMissingBean` resolves. Using `@Order` on an auto-configuration class does nothing useful. This is a small question that reliably separates people who have written a starter from people who have only read about them.

Q15. What is a `FailureAnalyzer` ?

The component behind Boot's readable startup errors — the box that says *"Description: Field x required a bean of type y that could not be found. Action: consider defining..."* rather than a raw stack trace. You can register your own for your starter's failure modes:

```
public class ApiKeyFailureAnalyzer extends AbstractFailureAnalyzer<MissingApiKeyException>
{
    @Override
    protected FailureAnalysis analyze(Throwable rootFailure, MissingApiKeyException cause)
    {
        return new FailureAnalysis(
            "No API key configured for the Acme client.",
            "Set acme.client.api-key in application.yml or ACME_CLIENT_API_KEY.",
            cause);
    }
}
```

Registered in `META-INF/spring.factories` under `org.springframework.boot.diagnostics.FailureAnalyzer`. Mentioning this unprompted when asked "how do you make a library pleasant to consume" lands well.

Q16. How does Spring Boot's modularization in 4.0 change the auto-configuration story?

Spring Boot 4.0 split the codebase into **70+ focused jars**, named `spring-boot-<technology>` with root packages `org.springframework.boot.<technology>`. Practically:

- Auto-configuration classes moved packages — for example `@EntityScan` now lives in `org.springframework.boot.persistence.autoconfigure`.
- Some things that used to arrive transitively now need an explicit dependency. Flyway, H2 and several test dependencies are the ones most commonly reported as breaking on day one.
- Several starters were renamed: `spring-boot-starter-web` → `spring-boot-starter-webmvc`, `spring-boot-starter-web-services` → `spring-boot-starter-webservices`, `spring-boot-starter-aop` → `spring-boot-starter-aspectj`, and the OAuth2 starters gained a `security-` prefix (`spring-boot-starter-security-oauth2-client`).

The *reason* is the interesting part for an interview: smaller, single-purpose jars mean a smaller classpath, fewer conditions to evaluate, faster startup, and a much better story for AOT and native images, where dead code on the classpath is genuine cost. See section 11 for the full 4.x picture.

3. Configuration, Profiles & Externalized Config

Q17. What is the property resolution order in Spring Boot?

`Environment` holds an ordered list of `PropertySource`s and the **first one that has the key wins**. From highest precedence downward, the ones that matter in practice:

1. Devtools global settings (`~/.config/spring-boot`), dev only
2. `@TestPropertySource` and `@SpringBootTest(properties = ...)` in tests
3. Command line arguments (`--server.port=8081`)
4. `SPRING_APPLICATION_JSON` (inline JSON in an env var)
5. `ServletConfig` / `ServletContext` init params
6. JNDI attributes
7. Java System properties (`-Dserver.port=8081`)
8. OS environment variables
9. Profile-specific config **outside** the jar
10. Profile-specific config **inside** the jar
11. Plain `application.yml` **outside** the jar
12. Plain `application.yml` **inside** the jar
13. `@PropertySource` on `@Configuration` classes
14. Default properties (`SpringApplication.setDefaultProperties`)

Two rules capture the philosophy: **more specific beats less specific**, and **more external beats more internal**. That is what makes the same artifact deployable to every environment — the twelve-factor promise.

Worth knowing: **relaxed binding** means `spring.datasource.url`, `SPRING_DATASOURCE_URL`, `spring.datasource.URL` and `spring-datasource-url` all bind to the same property. Environment variables are the reason it exists — most shells will not accept dots in a variable name.

Q18. `@Value` vs `@ConfigurationProperties` ?

`@Value("${x}")` injects a single value with SpEL support. `@ConfigurationProperties` binds a whole tree onto a typed object. For anything beyond one or two values, the typed binding wins:

```
@ConfigurationProperties(prefix = "acme.http")
@Validated
public record HttpProperties(
    @NotBlank String baseUrl,
    @DefaultValue("5s") Duration connectTimeout,
    @DefaultValue("30s") Duration readTimeout,
    @Min(1) @Max(100) int maxConnections,
    Map<String, String> defaultHeaders) { }
```

	@Value	@ConfigurationProperties
Relaxed binding	No	Yes
Type-safe / structured	No	Yes
JSR-303 validation	No	Yes, with @Validated
Meta-data for IDE	No	Yes, via the config processor
SpEL	Yes	No
Fails fast on bad config	No — fails at injection	Yes — at startup

That last row is the real argument. A typo in a `@Value` may only surface when that bean is first used; a `@ConfigurationProperties` binding failure kills startup with a precise message, which is exactly what you want in a deployment pipeline.

Note the `Duration` binding: `5s`, `100ms`, `2m` are parsed natively, as are `DataSize` values like `10MB`. Candidates who hand-roll `@Value("${timeout.millis}") long` lose a small point.

Q19. How do profiles work, and how should they be used in production?

`spring.profiles.active` selects profiles; `@Profile("x")` conditionally registers beans; `application-{profile}.yml` layers profile-specific properties over the base file. `spring.profiles.group` lets one profile activate several:

```
spring:
  profiles:
    group:
      prod: [ prod-db, prod-cache, metrics ]
```

The senior view is **restraint**. Profiles are excellent for swapping *infrastructure implementations* — an in-memory queue locally, SQS in production — and for test setup. They are dangerous when they start encoding *business* differences, because then your production code path is one that never runs in any lower environment. The rule of thumb: profiles change *where* things connect, configuration properties change *how* they behave, and feature flags change *what* users see. Conflating the three is how you get "works in staging" incidents.

Also: `@Profile("!prod")` is a perfectly good guard for dev-only endpoints, but never rely on it as a *security* control — a misconfigured `SPRING_PROFILES_ACTIVE` then becomes a breach rather than a bug.

Q20. How do you handle secrets?

Layered answer, from acceptable to good:

- **Never in the repo**, and never in `application.yml` that is baked into the image.
- **Environment variables** are the baseline. They are visible in `/proc`, in crash dumps, and in `/actuator/env` unless sanitized — Boot sanitizes keys matching `password`, `secret`, `key`, `token` and similar by default, which you should not weaken.
- **Config trees** are Boot's native answer for Kubernetes secrets mounted as files: `spring.config.import=optional:configtree:/etc/secrets/`, where each file's name is the property key and its contents the value. This avoids env-var exposure entirely.
- **A secrets manager** — Vault, AWS Secrets Manager, GCP Secret Manager — via `spring.config.import=vault://...`. The advantage is rotation without redeploy.
- **Encrypted at rest in Git** (SOPS, sealed-secrets) if you must keep config in the repo.

Mention `/actuator/env` sanitization and secret *rotation* — the ability to change a credential without a deploy is what distinguishes a real answer from a checklist.

Q21. What does `spring.config.import` give you that `@PropertySource` doesn't?

`@PropertySource` is a legacy, compile-time, `.properties`-only mechanism that ignores profiles and sits low in the precedence order. `spring.config.import` is the modern, runtime, extensible one:

```
spring:
  config:
    import:
      - optional:file:./local-overrides.yml
      - configtree:/etc/secrets/
      - vault://secret/my-app
      - aws-parameterstore:/prod/my-app/
```

It supports YAML, profile-specific files, `optional:` prefixes, and pluggable `ConfigDataLocationResolvers` — which is how Spring Cloud Config, Vault, and Consul integrate cleanly rather than hacking the environment during bootstrap. If a candidate still reaches for `bootstrap.yml`, that is a sign of pre-2020 habits: the bootstrap context was superseded by `spring.config.import` in Spring Cloud 2020.0.

Q22. How do you change configuration without a restart?

Three tiers, and the honest answer names the cost of each:

@RefreshScope + Spring Cloud Config. Beans in refresh scope are proxies that are discarded and rebuilt when `/actuator/refresh` is called. Works well; the catch is that the refresh is not atomic across a cluster and half-refreshed state is a real failure mode.

Feature flags (LaunchDarkly, Unleash, or a database table). Better for anything that toggles behaviour, because they are designed for gradual rollout, targeting and instant kill switches — and they do not require the property plumbing to be reloadable.

Rolling restart. Unfashionable, but with fast startup and a proper readiness probe it is often the simplest correct answer, and it keeps configuration immutable per instance — which makes debugging dramatically easier, since an instance's behaviour is a function of its deployment rather than of what happened to it since.

The strongest response acknowledges that dynamic configuration trades reproducibility for speed, and picks per use case rather than dogmatically.

4. REST API Design

Q23. How do you version a REST API?

Name the four options and the trade-offs, then commit to one.

URI path (`/api/v1/orders`) — most common, trivially visible in logs and caches, but it violates the idea that a URI identifies a resource, and it forces clients to rewrite URLs to upgrade.

Query parameter (`/api/orders?version=1`) — easy, but noisy and easily lost in caching layers.

Custom header (`X-API-Version: 1`) — keeps URIs clean, but is invisible in a browser and awkward to test by hand.

Content negotiation (`Accept: application/vnd.acme.order.v1+json`) — the most RESTful and the most precise, since it versions the *representation* rather than the resource; the cost is verbosity and unfamiliarity.

Spring Framework 7 / Boot 4 added **first-class API versioning**, so this is no longer hand-rolled:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @GetMapping(value =("/{id}", version = "1.0")
    public OrderV1 getV1(@PathVariable String id) { ... }

    @GetMapping(value =("/{id}", version = "2.0")
    public OrderV2 getV2(@PathVariable String id) { ... }
}
```

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer.useRequestHeader("X-API-Version") // or .usePathSegment(1)
                .setDefaultVersion("1.0")
                .addSupportedVersions("1.0", "2.0");
    }
}
```

The framework handles resolution, validation of unsupported versions, and deprecation headers.

The senior point, which matters more than the mechanism: the cheapest version is the one you never create. Additive, backward-compatible changes — new optional fields, new endpoints — need no version at all, provided clients are tolerant readers. Reserve

versioning for genuinely breaking changes, publish a deprecation policy with dates, emit `Deprecation` and `Sunset` headers, and instrument usage per version so you know when it is safe to remove one.

Q24. How should errors be returned?

Use **RFC 9457 Problem Details** (formerly RFC 7807) — it is a standard, and Spring has native support via `ProblemDetail` since Framework 6.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail handleNotFound(OrderNotFoundException ex) {
        var pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setType(Uri.create("https://api.acme.com/problems/order-not-found"));
        pd.setTitle("Order not found");
        pd.setProperty("orderId", ex.getOrderId());
        pd.setProperty("traceId", MDC.get("traceId"));
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        ProblemDetail pd = ProblemDetail.forStatus(HttpStatus.BAD_REQUEST);
        pd.setTitle("Validation failed");
        pd.setProperty("errors", ex.getBindingResult().getFieldErrors().stream()
            .map(fe -> Map.of("field", fe.getField(),
                "message", fe.getDefaultMessage()))
            .toList());
        return pd;
    }
}
```

Extending `ResponseEntityExceptionHandler` gives you Problem Detail responses for Spring's own exceptions too.

Principles to state: a **stable machine-readable code** (`type`) that clients can branch on, because they must never parse your human-readable message; a `traceId` so a support ticket maps to a log search; **never leak internals** — no stack traces, no SQL, no class names; and be careful that error messages do not become an enumeration oracle (`"user not found"` vs `"wrong password"` tells an attacker which emails are registered).

Q25. How do you make an endpoint idempotent?

`GET`, `PUT`, `DELETE` are idempotent by specification; `POST` is not. For the classic "the client retried and we charged them twice" problem, use an **idempotency key**:

```

@PostMapping("/payments")
public ResponseEntity<Payment> create(
    @RequestHeader("Idempotency-Key") @NotBlank String key,
    @Valid @RequestBody PaymentRequest request) {

    Optional<Payment> existing = idempotencyStore.find(key);
    if (existing.isPresent()) {
        return ResponseEntity.ok(existing.get()); // replay the original response
    }
    Payment payment = paymentService.process(request);
    idempotencyStore.save(key, payment, Duration.ofHours(24));
    return ResponseEntity.status(HttpStatus.CREATED).body(payment);
}

```

The details that show you have actually built one: the key must be stored **in the same transaction** as the side effect, or you have a race where the charge succeeds and the record does not; a unique constraint on the key column is what makes concurrent duplicates fail safely; you should also hash the request body and reject a reused key with a *different* payload (422); and the store needs a TTL so it does not grow forever.

Q26. RestTemplate , WebClient , or HTTP interfaces?

`RestTemplate` is in maintenance mode — not deprecated, but no new features. `WebClient` is the reactive, non-blocking client and works fine in a blocking app via `.block()`, though pulling in WebFlux for it is a real dependency cost. `RestClient` (Framework 6.1+) gives the fluent `WebClient` API with blocking semantics and no WebFlux dependency — usually the right default for an MVC app.

Best of all for a service client, declare the interface and let Spring implement it:

```

@HttpExchange("/orders")
public interface OrderClient {
    @GetExchange("/{id}")
    Order get(@PathVariable String id);

    @PostExchange
    Order create(@RequestBody OrderRequest request);
}

// Spring Boot 4: registered declaratively, no factory boilerplate
@Configuration
@ImportHttpServices(group = "orders", types = OrderClient.class)
class ClientConfig { }

```

```

spring:
  http:
    client:
      service:
        orders:
          base-url: https://orders.internal
          connect-timeout: 2s
          read-timeout: 5s

```

Always name timeouts unprompted. A client with no read timeout is the single most common cause of cascading failure — one slow dependency exhausts your thread pool and takes down a service that is itself perfectly healthy.

Q27. How do you paginate correctly?

Offset pagination (`?page=3&size=20`) is what `Pageable` gives you. It is simple, allows random access to any page, and degrades badly: `OFFSET 100000` makes the database walk and discard 100,000 rows, and rows shifting between requests cause items to be skipped or repeated.

Cursor (keyset) pagination is what you want for large or live datasets:

```
// WHERE (created_at, id) < (:cursorAt, :cursorId)
// ORDER BY created_at DESC, id DESC LIMIT :size
public Slice<Order> after(Instant createdAt, UUID id, int size);
```

Constant time regardless of depth, and stable under concurrent inserts. The costs are no random page access and no total count, plus the sort key must be unique — hence the composite `(created_at, id)` tiebreaker.

Note also that `Page<T>` issues a second `COUNT(*)` query; `Slice<T>` does not. If the UI only needs "next", returning `Slice` is free performance.

Q28. Where does validation belong?

`@Valid` on the controller parameter validates the request DTO — that is input validation, and it is the right place for format concerns: not blank, within range, matches a pattern. Domain invariants ("an order cannot ship before it is paid") belong in the domain model, not in annotations, because they must hold no matter which entry point mutated the object.

```
public record CreateOrderRequest(
    @NotBlank String customerId,
    @NotEmpty @Valid List<LineItem> items,           // @Valid cascades into nested
    @Future Instant requestedDelivery) {

    public record LineItem(@NotBlank String sku, @Positive int quantity) { }
}
```

Groups let one DTO validate differently per operation (`@Validated(OnCreate.class)`), and a custom `ConstraintValidator` covers rules the built-ins do not. For `@RequestParam`/`@PathVariable` validation you need `@Validated` **on the class** — a common gotcha, because those constraints are enforced by a method-level AOP proxy rather than by argument resolution.

One caveat worth raising: a validator that hits the database (checking uniqueness, say) is a constraint you cannot actually guarantee — between the check and the insert, another

request can win. Validate for a good error message, but let the unique index be the real enforcement.

5. Data Access, JPA & Transactions

Expect the most pointed questions here. Transaction semantics and the N+1 problem are where production incidents actually come from.

Q29. What is the N+1 select problem and how do you fix it?

One query fetches N parents; accessing a lazy association then triggers one query per parent — N+1 round trips. It usually appears only in production, because with ten rows of test data nobody notices.

```
List<Order> orders = orderRepository.findAll();           // 1 query
for (Order o : orders) {
    o.getLineItems().size();                             // N queries
}
```

Fixes, and when each applies:

```
// 1. JOIN FETCH - precise, per-query
@Query("select distinct o from Order o join fetch o.lineItems where o.status = :status")
List<Order> findByStatusWithItems(@Param("status") OrderStatus status);

// 2. Entity graph - declarative, reusable, composes with derived queries
@EntityGraph(attributePaths = { "lineItems", "customer" })
List<Order> findByStatus(OrderStatus status);

// 3. Batch fetching - one query per BATCH of parents instead of per parent
@BatchSize(size = 50) // on the collection or the entity
private List<LineItem> lineItems;
```

```
spring.jpa.properties.hibernate.default_batch_fetch_size: 50 # global, very effective
```

4. Projections. Often the real fix is not to load entities at all — if the endpoint returns five fields, select five fields:

```
public interface OrderSummary { // interface projection
    String getId();
    BigDecimal getTotal();
    String getCustomerName();
}
List<OrderSummary> findByStatus(OrderStatus status);
```

Two traps to mention: **JOIN FETCH with pagination** makes Hibernate fetch everything into memory and paginate there — it warns `HHH90003004: firstResult/maxResults specified with collection fetch`; use batch fetching or two queries instead. And **fetching two collections** in one query produces a cartesian product. **EAGER** on an association is almost never the answer — it makes the problem global and unremovable.

How you detect it is the part interviewers actually want: `spring.jpa.show-sql` for local work, `datasource-proxy` or Hibernate statistics in tests, and a test that asserts query counts so a regression fails CI rather than a dashboard.

Q30. What exactly does `@Transactional` do, and what are its pitfalls?

It wraps the method in a proxy that opens a transaction, commits on normal return, and rolls back on an unchecked exception. The pitfalls are a reliable interview sequence:

Self-invocation does nothing. Covered in Q7 — the call never passes through the proxy.

Checked exceptions do not roll back by default. Spring rolls back on `RuntimeException` and `Error`; a checked exception commits. Use `@Transactional(rollbackFor = Exception.class)` if you need otherwise. This surprises people constantly.

private, protected, and final methods are not advised. CGLIB cannot override them, and there is no warning — the annotation is silently ignored.

Transaction boundaries vs. HTTP boundaries. `@Transactional` on a controller keeps the transaction (and a pooled connection) open while you serialize the response. Keep transactions at the service layer and as short as possible.

Never do I/O inside a transaction. A remote HTTP call inside a transaction holds a database connection for the duration of someone else's outage. This is a top cause of pool exhaustion.

`readOnly = true` is not just a hint. It sets the JDBC connection read-only (which some databases and proxies use for routing to replicas) and puts Hibernate in `FlushMode.MANUAL`, skipping dirty checking — a genuine performance win on read paths.

```
@Service
public class OrderService {

    @Transactional(readOnly = true)
    public OrderView get(String id) { ... }

    @Transactional(timeout = 10, rollbackFor = Exception.class)
    public Order place(OrderRequest request) {
        Order order = repository.save(toOrder(request));
        // do NOT call the payment gateway here - publish after commit instead
        events.publishEvent(new OrderPlacedEvent(order.getId()));
        return order;
    }
}

@Component
class PaymentListener {
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    void on(OrderPlacedEvent e) { gateway.charge(e.orderId()); } // only if tx committed
}
```

That `@TransactionalEventListener(AFTER_COMMIT)` pattern is worth volunteering — it is the clean answer to "how do I avoid doing remote work inside a transaction while keeping it consistent".

Q31. Explain propagation levels.

Propagation	Behaviour
<code>REQUIRED</code> (default)	Join the current transaction, or start one
<code>REQUIRES_NEW</code>	Always start a new one; suspend the current
<code>SUPPORTS</code>	Join if present, otherwise run non-transactionally
<code>NOT_SUPPORTED</code>	Suspend any current transaction
<code>MANDATORY</code>	Must already be in one, else exception
<code>NEVER</code>	Must not be in one, else exception
<code>NESTED</code>	Savepoint within the current transaction

The two that get probed: `REQUIRES_NEW` is the right tool for audit logging that must survive the caller's rollback — but it consumes a **second connection** while the first is suspended, so a pool of 10 can deadlock with 10 concurrent callers. `NESTED` uses JDBC savepoints, so the inner work can roll back without killing the outer transaction; it needs a `DataSource` transaction manager and is not supported by JTA.

Follow-up: "You call a `REQUIRES_NEW` method and it throws. What happens?" The inner transaction rolls back; whether the outer one does depends on whether the exception propagates and is unchecked.

Q32. Optimistic vs pessimistic locking?

Optimistic — a `@Version` column; the update carries `WHERE version = ?`, and zero affected rows means somebody else won, producing `OptimisticLockException`. No database locks, scales well, and is the correct default when conflicts are rare. The application must handle the retry.

Pessimistic — `SELECT ... FOR UPDATE` via `@Lock(LockModeType.PESSIMISTIC_WRITE)`. Correct when contention is high and a retry loop would thrash, but it holds a row lock for the transaction's duration, so it serializes access and risks deadlock when lock order is inconsistent.

```

@Entity
public class Account {
    @Id private UUID id;
    @Version private long version;           // optimistic
    private BigDecimal balance;
}

public interface AccountRepository extends JpaRepository<Account, UUID> {
    @Lock(LockModeType.PESSIMISTIC_WRITE)
    @Query("select a from Account a where a.id = :id")
    Optional<Account> findByIdForUpdate(@Param("id") UUID id);
}

```

The senior addition: for a hot counter, the best answer is often **neither** — an atomic `UPDATE accounts SET balance = balance - :amt WHERE id = :id AND balance >= :amt` pushes the conflict resolution into the database and avoids read-modify-write entirely.

Q33. How do you size a connection pool?

The instinct is "bigger is better"; the correct answer is the opposite. HikariCP's own guidance, derived from the PostgreSQL project, is roughly `connections = ((core_count * 2) + effective_spindle_count)` — for most services that means **something like 10, not 100**. Beyond the point where the database can execute requests in parallel, more connections add context switching and lock contention, and throughput drops while latency climbs.

```

spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      minimum-idle: 10           # equal to max = fixed-size pool, avoids churn
      connection-timeout: 3000    # fail fast rather than queue forever
      max-lifetime: 1800000       # under any DB/proxy idle timeout
      leak-detection-threshold: 20000

```

Then the arithmetic that shows you have operated this: pool size × instance count must stay under the database's `max_connections`, and you must leave headroom for migrations, admin sessions and replicas. Twenty pods at 20 connections each is 400 connections — more than many managed Postgres instances allow by default. Also note that `connection-timeout` is what turns a saturated pool into a fast error instead of a slow hang, which is the difference between a degraded service and an outage.

Q34. `LazyInitializationException` — cause and correct fix?

Accessing a lazy association after the persistence context has closed. The transaction ended at the service boundary and the controller or serializer then touched `order.getLineItems()`.

The **wrong** fixes are the popular ones: `spring.jpa.open-in-view=true` (on by default, and it keeps the `EntityManager` and its connection open for the whole request — it hides the

problem and holds connections during view rendering) and switching associations to **EAGER** (global cost for a local problem).

The right fixes: fetch what you need inside the transaction using a fetch join or entity graph; map to a DTO **inside** the service while the context is open; or use a projection so entities never escape. Turning **open-in-view** off explicitly is a good practice — it converts a silent performance tax into a loud error at development time.

Q35. How do you manage schema migrations?

Flyway or **Liquibase**, never **ddl-auto** beyond **validate** in production. **spring.jpa.hibernate.ddl-auto** should be **validate** (or **none**) in every deployed environment; **update** is dangerous because it never drops or alters destructively, so schema and entity drift apart quietly, and it is non deterministic across versions.

Points that show operational maturity:

- Migrations are **forward-only and immutable** once merged — Flyway checksums enforce this.
- **Expand/contract** for anything breaking: add the new column, backfill, dual-write, switch reads, then drop the old column in a later release. This keeps the schema compatible with both the old and new application versions, which is what makes zero-downtime rolling deploys possible.
- **Long-running DDL locks tables.** Adding a **NOT NULL** column with a default rewrote the whole table in older PostgreSQL; creating an index needs **CONCURRENTLY**. Know the behaviour of your specific database version.
- Test migrations against a **real database** with Testcontainers, not H2 — dialect differences mean an H2-passing migration proves very little.
- Note for Boot 4: Flyway is one of the dependencies that modularization no longer pulls in transitively, so an upgrade may need it declared explicitly.

6. Spring Security & JWT

Q36. Explain the Spring Security filter chain.

Spring Security installs a single servlet `Filter` — `DelegatingFilterProxy`, registered as `springSecurityFilterChain` — which delegates to `FilterChainProxy`. That holds an ordered list of `SecurityFilterChain`s, each with a matcher; the **first matching chain wins** and its filters run.

The ones worth naming in order:

1. `SecurityContextHolderFilter` — loads any existing `SecurityContext` from the session
2. `CsrfFilter`
3. Authentication filters — `UsernamePasswordAuthenticationFilter`, `BearerTokenAuthenticationFilter`, `OAuth2`
4. `ExceptionTranslationFilter` — converts `AuthenticationException` → 401 entry point, `AccessDeniedException` → 403
5. `AuthorizationFilter` — the final gate that enforces the request-level rules

`SecurityContextHolder` holds the authenticated principal in a `ThreadLocal` by default — which is precisely why it needs care with `@Async`, and why `DelegatingSecurityContextExecutor` or `MODE_INHERITABLETHREADLOCAL` exists.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .securityMatcher("/api/**")
        .csrf(csrf -> csrf.disable()) // stateless + bearer tokens
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/public/**").permitAll()
            .requestMatchers("/api/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .build();
}
```

Multiple chains with `securityMatcher` is the idiomatic way to have a stateless API and a session-based admin UI in one application — and interviewers like that question because the naive answer (one chain with mixed rules) does not actually work.

Q37. Authentication vs authorization, and where does each happen?

Authentication establishes **who you are**; authorization decides **what you may do**. In Spring Security, authentication happens early in the chain and results in an `Authentication` in the `SecurityContext`; authorization happens at the end of the chain (`AuthorizationFilter`, using `authorizeHttpRequests` rules) and optionally again at the method level.

Request-level rules are coarse and URL-shaped; method security is fine-grained and travels with the code:

```
@EnableMethodSecurity // replaces the old @EnableGlobalMethodSecurity
public class SecurityConfig { }

@Service
public class DocumentService {

    @PreAuthorize("hasRole('EDITOR')")
    public void publish(UUID id) { }

    @PreAuthorize("@documentPermissions.canRead(#id, authentication.name)")
    public Document read(UUID id) { }

    @PostAuthorize("returnObject.ownerId == authentication.name")
    public Document load(UUID id) { }

    @PreFilter("filterObject.ownerId == authentication.name")
    public void bulkUpdate(List<Document> docs) { }
}
```

Say explicitly that **URL rules alone are insufficient** for anything with per-object ownership — `/documents/{id}` cannot be secured by path pattern, and that gap is the most common real-world authorization vulnerability (IDOR). Note also that `@PreAuthorize` uses the same proxy mechanism as `@Transactional`, so self-invocation bypasses it too.

Q38. How do you validate a JWT, and what do people get wrong?

With `oauth2ResourceServer().jwt()` Spring does it for you, which is the point — hand-rolled JWT filters are where the bugs live.

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.acme.com/realms/prod # JWKS discovery, checks iss
          # jwk-set-uri: https://auth.acme.com/.../certs # alternative if no discovery
```

```
@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    var authorities = new JwtGrantedAuthoritiesConverter();
    authorities.setAuthoritiesClaimName("roles");
    authorities.setAuthorityPrefix("ROLE_");
    var converter = new JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(authorities);
    return converter;
}
```

What goes wrong:

- **Not verifying the signature**, or trusting the token's own `alg` header — the `alg: none` and RS256→HS256 confusion attacks, where the attacker signs with the public key as an HMAC secret. Pin the expected algorithm.
- **Not validating `iss` and `aud`**. A valid token from another tenant or another application is still a valid signature. `JwtValidators.createDefaultWithIssuer` plus an audience validator.
- **No clock skew tolerance** causing spurious 401s across machines (60s is typical).
- **Treating JWTs as revocable**. They are not. A stolen token is valid until it expires, which is the fundamental trade-off of stateless auth.
- **Putting sensitive data in the payload** — it is base64, not encryption. Anyone can read it.
- **Long expiry**. Short-lived access tokens (5–15 min) plus a refresh token is the standard shape; the refresh token is revocable because it is checked against server state.

Follow-up: "How do you log a user out immediately?" You cannot, with pure JWT. Options: keep a revocation/denylist keyed by `jti` (reintroduces state, but only for revoked tokens), keep expiry very short so the window is small, or use opaque tokens with introspection.

Q39. How does CSRF work, and when can you disable it?

CSRF exploits the browser's habit of attaching cookies automatically: a malicious page submits a form to your domain and the browser includes the session cookie. The defence is a token the attacker cannot read or guess, submitted with the request.

Disable it **only** when your API is stateless and authenticated by a header (`Authorization: Bearer`), because the browser does not attach headers automatically, so the attack does not apply. If you authenticate with cookies — including a JWT stored in a cookie — **you need CSRF protection**. That last case is the one candidates get wrong most often.

For a SPA, the pattern is `CookieCsrfTokenRepository.withHttpOnlyFalse()` so JavaScript can read the token and echo it in a header. `SameSite=Lax` or `Strict` cookies are good defence in depth but not a complete substitute across all browsers and flows.

Q40. How do you store passwords?

`DelegatingPasswordEncoder` — the Boot default via `PasswordEncoderFactories.createDelegatingPasswordEncoder()`. Hashes are prefixed with the algorithm id (`{bcrypt}$2a$10$...`), so you can migrate algorithms without invalidating existing passwords: new hashes use the new default, old ones still verify.

```
@Bean
PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder(); // bcrypt by
    default
}
```

Say **bcrypt, scrypt, or Argon2** — deliberately slow, salted, memory-hard functions. Never MD5 or SHA-256, even salted: they are fast, which is exactly the wrong property. Tune the work factor so a hash takes on the order of 100ms on your hardware, and revisit it as hardware improves. Argon2id is the current recommendation for new systems.

Q41. What are the main security issues you would look for in a Spring Boot service?

A structured answer beats a list. Mine would be:

Authentication and session — token validation as in Q38, session fixation protection (on by default), secure/HttpOnly/SameSite cookie flags.

Authorization — object-level checks, not just URL rules; the IDOR gap from Q37.

Injection — parameterized queries only. `@Query` with string concatenation is SQL injection even inside JPA; `nativeQuery = true` with a concatenated sort column is the classic. Note that Spring Data derived queries and bound parameters are safe.

Mass assignment — binding a request body straight onto a JPA entity lets a caller set `role=ADMIN` or `id`. Use dedicated request DTOs; this is one of the strongest arguments against "just return the entity".

Exposure — actuator endpoints on a public port (`/actuator/env` , `/heapdump` are gold for an attacker), stack traces in responses, verbose error messages.

Dependencies — this is where most real CVEs come from. `mvn dependency-check` / Dependabot / Snyk in CI, and an actual process for acting on findings. Mention Log4Shell and Spring4Shell as why supply chain matters, and note Boot 4.1's new `InetAddressFilter` for **SSRF** protection when the application fetches user-supplied URLs.

Transport — TLS everywhere including internal hops, HSTS, and no secrets in logs.

7. Testing Strategy

Q42. What are test slices and why do they matter?

`@SpringBootTest` starts the whole application context. Slices start only the part you are testing, which is dramatically faster and produces failures that point at one layer.

Annotation	Loads	Typical use
<code>@WebMvcTest</code>	Controllers, filters, <code>@ControllerAdvice</code> , converters — no services or repositories	Controller behaviour, status codes, JSON, security rules
<code>@DataJpaTest</code>	JPA repositories, <code>EntityManager</code> , an embedded DB, transactional and rolled back	Query correctness
<code>@JdbcTest</code> , <code>@DataRedisTest</code> , <code>@DataMongoTest</code>	The equivalent for other stores	
<code>@JsonTest</code>	Jackson/Gson config only	Serialization contracts
<code>@RestClientTest</code>	<code>RestTemplate / RestClient</code> + <code>MockRestServiceServer</code>	Outbound client behaviour
<code>@WebFluxTest</code>	WebFlux controllers + <code>WebTestClient</code>	Reactive controllers

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean OrderService orderService; // Boot 3.4+; @MockBean gone in Boot 4

    @Test
    void returnsProblemDetailWhenMissing() throws Exception {
        given(orderService.get("1")).willThrow(new OrderNotFoundException("1"));

        mvc.perform(get("/orders/1"))
            .andExpect(status().isNotFound())
            .andExpect(header().string("Content-Type", "application/problem+json"))
            .andExpect(jsonPath("$.title").value("Order not found"));
    }
}
```

The senior framing: **context caching** is why this matters at scale. Spring caches contexts by configuration key, so every distinct combination of slices, properties, mocks and profiles creates another context. A suite with fifty subtly different configurations spends most of its time starting Spring. Keeping configurations uniform — and resisting one-off `@TestPropertySource` values — is often the single biggest test-suite speedup available.

Q43. Testcontainers vs H2 for repository tests?

Testcontainers, for one reason: H2 is not your production database. Dialect differences mean an H2-green test says little about Postgres behaviour — `JSONB`, arrays, window functions, upsert syntax, lock semantics, sequence behaviour and index types all differ, and Flyway migrations that pass on H2 routinely fail on the real thing.

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE) // do not swap in H2
@Testcontainers
class OrderRepositoryTest {

    @Container
    @ServiceConnection // Boot 3.1+: wires URL + credentials
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres:16-alpine");

    @Autowired OrderRepository repository;
}
```

`@ServiceConnection` removed the `@DynamicPropertySource` boilerplate and is worth naming. Mention **container reuse** (`testcontainers.reuse.enable=true`) and making the container `static` so one instance serves the class, since startup cost is the usual objection. For local development, Boot 3.1's `spring-boot-testcontainers` with `@TestConfiguration` lets you boot the *application* against containers too.

The honest trade-off: Testcontainers needs a working Docker daemon in CI and is slower than an in-memory database. The answer is that correctness beats speed for the data layer, and slices plus container reuse keep it manageable.

Q44. How do you test a `@Transactional` service method?

Careful here — this question has a trap. `@Transactional` **on the test** rolls back after each test, which is convenient for isolation but means you are not testing commit behaviour. Anything that happens at commit time — `@TransactionalEventListener(AFTER_COMMIT)`, deferred constraint checks, flush-time optimistic locking failures — will not fire.

For those cases, drop the test-level transaction and clean up explicitly:

```
@SpringBootTest
class OrderServiceIT {

    @Autowired OrderService service;
    @Autowired OrderRepository repository;

    @AfterEach void cleanup() { repository.deleteAll(); }

    @Test
    void publishesEventAfterCommit() {
        service.place(request());
        await().atMost(Duration.ofSeconds(2))
            .untilAsserted(() -> verify(gateway).charge(any()));
    }
}
```

Also worth saying: testing that "the annotation is present" is worthless. Test the *behaviour* — that a failure halfway through leaves no partial state. That test is the one that catches a misplaced `REQUIRES_NEW` or a swallowed exception.

Q45. How do you test asynchronous or scheduled code?

Do not use `Thread.sleep()`. Use **Awaitility** for polling assertions, as above. For scheduled jobs, the best design is to keep the schedule trivial and the work in a plain, directly testable method:

```
@Component
class ReconciliationJob {
    @Scheduled(cron = "0 0 2 * * *")
    void scheduled() { reconciler.run(Instant.now()); } // untested one-liner
}
// test reconciler.run(fixedInstant) directly, with a Clock bean for determinism
```

Injecting a `Clock` bean rather than calling `Instant.now()` inside business logic is the detail that makes time-dependent code testable, and it is a strong senior signal.

For `@Async`, note that the method must be called through the proxy (Q7) and that `SecurityContext` and MDC do **not** propagate to the new thread by default — `DelegatingSecurityContextExecutor` and a task decorator fix that, and it is a frequent production bug.

Q46. What is your view on mocking?

Mock at **architectural boundaries** — the HTTP client, the message broker, the payment gateway. Do not mock the class under test's own collaborators when they are cheap value objects or pure functions, and do not mock the repository *and* then also assert on the repository, which tests your mock rather than your code.

The failure mode to name is **over-mocking**: a test suite where every collaborator is a mock passes happily while the application is broken, because nothing verifies that the pieces agree. Mocks encode your *assumption* about a dependency's behaviour; when that assumption drifts from reality the test keeps passing. That is what integration tests and **contract tests** (Spring Cloud Contract, Pact) exist to prevent: the provider verifies the same contract the consumer stubbed against.

A reasonable shape to state: many fast unit tests for logic, a middle layer of slice tests for wiring, a small number of full integration tests for critical paths, and contract tests wherever two services meet.

Q47. What changed for testing in Spring Boot 4?

Several things that will bite on upgrade, so it makes a good "are you current" question:

- `@MockBean` and `@SpyBean` are **removed** — replaced by `@MockitoBean` and `@MockitoSpyBean` (introduced in Boot 3.4, so you can migrate before upgrading).

- `@SpringBootTest` **no longer auto-configures** `MockMvc`, `WebTestClient` or `TestRestTemplate`; add `@AutoConfigureMockMvc` or `@AutoConfigureTestRestTemplate` explicitly.
- `MockitoTestExecutionListener` was removed and `@PropertyMapping` moved to `org.springframework.boot.test.context`.
- Spock integration was dropped.
- Test-related dependencies are among those affected by modularization, so some may need declaring explicitly.

8. Microservices & Resilience Patterns

Q48. How do you decide service boundaries?

The answer interviewers want is **not** "by layer" or "by entity". Boundaries should follow **business capabilities and data ownership** — a bounded context in DDD terms. The test: a service owns its data exclusively, and a typical feature change touches one service.

Practical heuristics to offer:

- **If two services must always deploy together, they are one service.** Distributed monolith is the most common failure mode, and it costs you network calls, partial failure and eventual consistency while delivering none of the independence you paid for.
- **Follow the data.** If two "services" write the same table, you have one service with two deployment units.
- **Follow the team.** Conway's Law is descriptive, not aspirational — a boundary that cuts across a team's responsibilities will be eroded.
- **Start with a modular monolith.** Boundaries are hypotheses, and they are far cheaper to move inside one codebase than across repositories and wire protocols. Extract a service when there is a concrete reason: independent scaling, independent release cadence, a different reliability or compliance requirement, or team autonomy at a size where coordination is the bottleneck.

Saying "we would default to a modular monolith and extract under pressure" is a stronger senior answer than enthusiasm for microservices, provided you can justify when to split.

Q49. How do you make service-to-service calls resilient?

Layer the defences, and name what each protects against:

Timeouts — always, on every call. Protects your threads from their latency. **Retries with exponential backoff and jitter** — for transient faults only, and only on idempotent operations. Jitter matters: synchronized retries from many clients create a thundering herd. **Circuit breaker** — stops hammering a dependency that is already failing and fails fast instead, giving it room to recover. **Bulkhead** — separate pools per dependency so one slow dependency cannot consume all threads. **Fallback** — cached or degraded responses where the business can tolerate them.

Spring Framework 7 / Boot 4 ships **core resilience annotations**, so simple cases no longer need Resilience4j:

```

@Service
public class InventoryClient {

    @Retryable(maxAttempts = 3, delay = 100, multiplier = 2.0, jitter = 50,
              includes = TransientException.class)
    public Stock check(String sku) { return client.get(sku); }

    @ConcurrencyLimit(20) // bulkhead: at most 20 concurrent calls
    public void reindex() { ... }
}

```

For circuit breaking and richer policies, **Resilience4j** remains the tool:

```

@CircuitBreaker(name = "inventory", fallbackMethod = "fallback")
@Bulkhead(name = "inventory")
@TimeLimiter(name = "inventory")
public Stock check(String sku) { return client.get(sku); }

private Stock fallback(String sku, Throwable t) {
    return Stock.unknown(sku); // degrade, don't fail the whole page
}

```

Then the point that distinguishes experience: **retries amplify load during an incident**. Three retries per client turns a struggling dependency's traffic into 3×. Retry budgets, circuit breakers ahead of retries, and never retrying a non-idempotent write are what keep a degradation from becoming an outage.

Q50. How do you handle distributed transactions?

You do not — two-phase commit across services is a trap: it couples availability (any participant down blocks everyone), scales poorly, and most modern datastores and brokers do not support it well. Instead:

Saga — a sequence of local transactions, each with a compensating action. Choreography (each service reacts to events) is simple for short flows but the overall process exists nowhere explicitly. Orchestration (a coordinator drives the steps) is easier to reason about, test and observe, at the cost of a component that knows the flow. For anything beyond three steps, orchestration usually wins.

Transactional outbox — the standard fix for the dual-write problem, and the answer to "how do you atomically update the database *and* publish an event":

```

@Transactional
public void placeOrder(OrderRequest request) {
    Order order = orderRepository.save(toOrder(request));
    outboxRepository.save(new OutboxEvent(
        "order.placed", order.getId(), toJson(order))); // same transaction, same DB
}
// A separate poller (or CDC via Debezium) reads the outbox and publishes to Kafka.

```

Both the state change and the event record commit together, so you cannot end up with one without the other. Delivery is then **at-least-once**, which means consumers must be **idempotent** — dedupe on an event id. Saying "at-least-once plus idempotent consumers" unprompted is a strong signal; exactly-once delivery does not exist end-to-end, only effectively-once processing.

Also worth naming: compensations are not rollbacks. Refunding a charge is a new business fact, not an undo, and some actions (an email sent) cannot be compensated at all — which should influence the order of steps in the saga: do the reversible, cheap things first.

Q51. Sync or async between services?

Synchronous (REST/gRPC) when the caller needs the result to continue — a price quote, an authorization check. Simple to reason about and debug; the cost is temporal coupling, so the caller's availability is the product of everything downstream.

Asynchronous (Kafka, RabbitMQ, SQS) when the work can proceed independently — notifications, projections, analytics, most "and then also" logic. Buys decoupling, natural buffering under load and independent scaling; costs you eventual consistency, harder debugging, ordering concerns, and the operational weight of a broker.

The senior instinct: **commands synchronous, events asynchronous**, and be suspicious of any synchronous chain more than two hops deep — latency and failure probability compound multiplicatively. If service A calls B calls C calls D, A's availability is the product of four numbers, and a 99.9% service becomes 99.6%.

Boot 4.1 added first-class **Spring gRPC** support, which is worth mentioning for internal service-to-service calls where schema and performance matter more than browser friendliness.

Q52. How do you do service discovery and config in Kubernetes?

Be careful not to over-answer with Netflix-era tooling. In Kubernetes you often need **neither Eureka nor Config Server**: Services give you DNS-based discovery and client-side load balancing is handled by kube-proxy or a service mesh, while ConfigMaps and Secrets mount as files that `spring.config.import=configtree:` reads natively.

Reach for Spring Cloud when you have a genuine gap: `spring-cloud-kubernetes` for watching ConfigMaps and triggering `@RefreshScope`, Spring Cloud Gateway for an application-aware edge, Config Server when you need versioned config shared across many services and environments outside Kubernetes.

Saying "the platform already solves this" where true is a sign of judgment; adding Eureka inside Kubernetes is a classic case of paying for a solved problem twice.

Q53. How do you keep data consistent across services that need each other's data?

Options, roughly in increasing order of coupling:

Replicate read models via events. Each service keeps its own denormalized copy, updated from the owner's events. Fast reads, no runtime coupling, eventually consistent. This is the default.

Query the owner synchronously. Always fresh, but adds latency and a hard dependency; acceptable for low-volume, high-consistency reads.

API composition at the edge — a gateway or BFF fans out and joins. Keeps services clean; the join logic and its failure modes move to the edge.

Shared database. Almost always wrong; it removes the boundary entirely.

The thing to volunteer: *most business processes tolerate staleness far better than engineers assume, but the tolerance must be an explicit decision.* "This dashboard may be 30 seconds stale, and that is fine; the fraud check must not be" is the conversation to have, and it usually pushes you toward events for the many and synchronous calls for the few.

9. Performance, Concurrency & Scaling

Q54. What are virtual threads and when should you use them?

Virtual threads (JEP 444, final in Java 21) are JVM-managed threads scheduled onto a small pool of carrier threads. When a virtual thread blocks on I/O it is **unmounted** rather than parking an OS thread, so a service can have hundreds of thousands of concurrent requests in flight with a handful of platform threads.

```
spring.threads.virtual.enabled: true    # Boot 3.2+: Tomcat and @Async use virtual threads
```

What this buys you: the **throughput of async with the readability of blocking code**. Stack traces stay meaningful, debuggers work, `try/finally` works, and thread-per-request stays the mental model. For most I/O-bound Spring MVC applications this is a bigger practical win than rewriting in WebFlux.

What to watch for, and interviewers will ask:

- **Pinning.** A virtual thread inside a `synchronized` block that blocks cannot unmount, pinning the carrier thread. JDK 24 (JEP 491) largely removed this limitation, but on Java 21 it matters — prefer `ReentrantLock` in hot paths, and older library code may still pin.
- **ThreadLocal costs.** Cheap per thread, but with a million threads the arithmetic changes. Scoped values are the successor.
- **Pooling becomes pointless** — virtual threads are cheap to create; never pool them.
- **Your bottleneck may not move.** If the constraint is a 10-connection database pool, letting 100,000 requests arrive at it faster just moves the queue. Virtual threads remove the *thread* bottleneck, not every bottleneck — and that answer shows systems thinking.

Boot 4.1 added **lazy JDBC connection acquisition**, which pairs well here: connections are taken only when actually needed rather than at transaction start, reducing pool pressure on paths that may not touch the database.

Q55. WebFlux or MVC?

Default to **MVC**, especially now that virtual threads close most of the throughput gap. Choose WebFlux when you have a specific reason:

- Very high concurrency with mostly I/O-bound work and strict memory limits
- **Streaming** semantics — server-sent events, long-lived connections, backpressure that must propagate to the producer
- The whole call chain is already reactive (R2DBC, reactive Kafka, reactive Redis)

The costs are real and worth naming: a steep learning curve, stack traces that lose the logical flow, debugging that is materially harder, `ThreadLocal`-based tooling (MDC, `SecurityContextHolder`, transactions) needing reactive equivalents via `Context`, and — the

killer — **one blocking call anywhere poisons the event loop**. A single JDBC call in a reactive chain can stall all requests on that thread.

The mature framing: reactive is a **backpressure** and resource-efficiency tool, not a "make it faster" tool. If you do not need backpressure and you are not memory constrained, MVC plus virtual threads gives you most of the benefit for a fraction of the complexity.

Q56. How do you improve Spring Boot startup time?

Measure first — `-Ddebug` or the `ApplicationStartup` metrics give you the actual breakdown:

```
SpringApplication app = new SpringApplication(MyApp.class);  
app.setApplicationStartup(new BufferingApplicationStartup(2048)); // GET /actuator/startup
```

Then, roughly in order of payoff:

- **Trim the classpath**. Every jar is more auto-configuration to evaluate. Boot 4's modularization helps directly here.
- **Narrow component scanning**; avoid scanning broad packages.
- **Lazy initialization** (`spring.main.lazy-initialization=true`) — fast startup, but it moves the cost to first request and turns startup failures into runtime failures. Good for local development, questionable for production.
- **Exclude unused auto-configurations** explicitly.
- **CDS / AppCDS** — class-data sharing gives a solid win for little effort, and Boot 3.3+ has first-class support (`java -XX:ArchiveClassesAtExit=app.jsa`).
- **Spring AOT + GraalVM native image** — the big one: startup drops from seconds to tens of milliseconds and memory falls substantially.

For native images, state the trade-offs honestly: a closed-world assumption means reflection, dynamic proxies and resource loading must be known at build time (hints required); build times go from seconds to minutes; peak throughput is typically *lower* than the JVM's because there is no JIT profile-guided optimization; and debugging tooling is weaker. Native is compelling for scale-to-zero serverless and high-density deployments, and usually unnecessary for a long-running service where a few seconds of startup is irrelevant.

Q57. How do you add caching, and what are the traps?

```

@EnableCaching
@Configuration
class CacheConfig {
    @Bean
    RedisCacheConfiguration cacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues();
    }
}

@Service
public class ProductService {

    @Cacheable(value = "products", key = "#id", unless = "#result == null")
    public Product get(String id) { return repository.findById(id).orElse(null); }

    @CacheEvict(value = "products", key = "#product.id")
    public Product update(Product product) { return repository.save(product); }
}

```

Traps worth raising unprompted:

- **Self-invocation** again — `@Cacheable` is proxy-based, same as Q7.
- **Cache stampede.** When a hot key expires, every concurrent request misses and hits the database at once. `sync = true` helps within one JVM; across a cluster you need a lock, probabilistic early expiry, or refresh-ahead.
- **Invalidation correctness.** `@CacheEvict` after a database write leaves a window where another node repopulates stale data. For strict cases, evict *after commit* (`@TransactionalEventListener`) or accept a short TTL as the real safety net.
- **Caching nulls** — either cache them deliberately to prevent a lookup storm for missing keys, or disable it deliberately. Doing it by accident is how you cache a transient failure.
- **What not to cache:** anything where staleness is a correctness problem — permissions, balances, inventory counts at checkout.

Always ask *why* the call is slow before caching it. A cache in front of an N+1 query is a workaround; fixing the query is the fix.

Q58. A production endpoint got slow. Walk me through your diagnosis.

A structured, evidence-first answer matters more than any specific tool:

1. Scope it. All endpoints or one? All instances or one? Correlated with a deploy, a traffic change, or a dependency's incident? Sudden or gradual? Check the RED metrics (rate, errors, duration) and compare percentiles — a p99 change with a flat p50 is a different problem from both moving.

2. Look at traces before guessing. A distributed trace for a slow request usually shows the answer directly: time in the database, time in a downstream call, or time unaccounted for (which means queueing or GC).

3. The usual culprits, roughly by frequency: a query that lost its index or an N+1 that grew with data volume; connection pool saturation (queue time, not execution time); a downstream dependency slowing down; GC pressure from an unbounded result set or cache; thread pool exhaustion from a missing timeout; a lock or hot row serializing writes.

4. Instrument, don't speculate. `@Timed` on suspects, slow-query logs, `pg_stat_statements`, async-profiler or JFR for CPU and allocation, and heap analysis for retention.

5. Fix, then prevent. Add the index *and* the query-count test; add the timeout *and* the alert on pool saturation.

Naming the diagnostic order — metrics → traces → profiler → code — is what separates a senior answer from a list of tools.

Q59. How do you scale a Spring Boot service?

Vertical first, because it is free: JVM heap sizing, `-XX:MaxRAMPercentage` in containers (the JVM does respect cgroup limits since Java 10, but the default 25% heap is often wrong), the right garbage collector — G1 for balanced workloads, ZGC for low pause at large heaps.

Horizontal, which requires the application to be **stateless**: no in-memory sessions (externalize to Redis via Spring Session), no local scheduler assuming singleton execution (use `ShedLock` or a distributed scheduler), no local file state, no sticky-session assumptions.

Then the parts people forget, which is where the senior answer lives: **the database is usually the real limit**. Adding pods multiplies connections against a fixed `max_connections`, so you need connection proxies (PgBouncer), read replicas with explicit read/write routing, caching, and eventually partitioning or sharding. And scaling requires **graceful shutdown** (`server.shutdown=graceful`) plus correct readiness probes, or every scale-down event drops in-flight requests — Boot 4 enables liveness and readiness probes by default, which helps.

10. Observability & Production Readiness

Q60. What are the three pillars, and what does Spring Boot give you for each?

Metrics — aggregated numbers, cheap to store, good for alerting and trends. Micrometer is the facade (a "SLF4J for metrics") with registries for Prometheus, OTLP, Datadog and others.

Logs — discrete events with detail, expensive at volume, good for forensics.

Traces — the causal path of one request across services, the only thing that answers "where did the time go" in a distributed system. Micrometer Tracing bridges to OpenTelemetry or Brave.

Since Boot 3, the story consolidated on Micrometer + OpenTelemetry:

```
<dependency><groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId></dependency>
<dependency><groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId></dependency>
<dependency><groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-otel</artifactId></dependency>
```

```
management:
  endpoints.web.exposure.include: health,info,metrics,prometheus
  endpoint.health.show-details: when-authorized
  metrics.tags.application: ${spring.application.name}
  tracing.sampling.probability: 0.1
```

The fourth pillar worth mentioning: **continuous profiling** (async-profiler, JFR, Pyroscope), which answers "which code burned the CPU" in a way the other three cannot.

Q61. What makes a good health check?

Distinguish the two probes, because conflating them causes outages:

Liveness — "is this process broken beyond recovery?" Failing it **restarts the pod**. It must not depend on external systems. If liveness checks the database, a database blip restarts every pod in the fleet simultaneously, turning a degradation into a full outage.

Readiness — "should this instance receive traffic right now?" Failing it removes the pod from the load balancer without killing it. This is where dependency checks belong — but only for dependencies without which the service genuinely cannot serve.

```
management:
  endpoint.health.probes.enabled: true           # /actuator/health/liveness and /readiness
  health.livenessstate.enabled: true
  health.readinessstate.enabled: true
```

```

@Component
public class QueueHealthIndicator implements HealthIndicator {
    @Override
    public Health health() {
        long depth = queue.depth();
        return depth < 10_000
            ? Health.up().withDetail("depth", depth).build()
            : Health.down().withDetail("depth", depth).build();
    }
}

```

Senior points: set `management.endpoint.health.group.readiness.include` deliberately rather than letting every auto-registered indicator gate your traffic; a slow health check is itself an outage risk, so cache or time-bound expensive ones; and health endpoints should be on a separate management port, not exposed publicly. Boot 4 enables liveness and readiness probes by default.

Q62. How does distributed tracing work, and what is context propagation?

A trace is a tree of spans sharing a trace id. Each service creates spans and propagates the context to downstream calls via headers — **W3C Trace Context** (`traceparent`) is the standard, replacing the older B3 headers.

Spring instruments this automatically for `RestClient`, `WebClient`, `RestTemplate`, Kafka listeners and scheduled tasks, provided the beans are created from Boot's auto-configured builders — a hand-constructed `RestTemplate` silently loses propagation, which is a classic "why is my trace broken" bug.

Put the trace id in logs so a trace and its logs join up:

```

logging.pattern.level: "%5p [%${spring.application.name:},%X{traceId:-},%X{spanId:-}]"

```

Sampling is the real-world concern: 100% is unaffordable at volume, so head-based sampling at 1-10% is typical, with tail-based sampling (decide after seeing the whole trace) where the backend supports it, so that errors and slow requests are always kept.

Custom spans via the `Observation` API, which produces metrics *and* traces from one instrumentation:

```

@Service
class PricingService {
    private final ObservationRegistry registry;

    Price calculate(Order order) {
        return Observation.createNotStarted("pricing.calculate", registry)
            .lowCardinalityKeyValue("channel", order.channel()) // safe metric tag
            .highCardinalityKeyValue("orderId", order.id())      // trace-only
            .observe(() -> doCalculate(order));
    }
}

```

The low/high cardinality distinction is worth volunteering — it is exactly the mistake in Q63.

Q63. What is metric cardinality and why does it matter?

Every unique combination of tag values creates a separate time series. Tagging with a user id, order id, or raw URL path explodes cardinality and can take down your metrics backend — the classic incident is `uri` tagged with `/orders/12345` instead of the templated `/orders/{id}`, producing a new series per order.

Rules: tags must be **bounded and low cardinality** (status, method, templated route, region); identifiers belong in **traces and logs**, not metric tags. Micrometer's `MeterFilter` can cap this defensively:

```
@Bean
MeterFilter cardinalityGuard() {
    return MeterFilter.maximumAllowableTags("http.server.requests", "uri", 100,
        MeterFilter.deny());
}
```

Q64. How would you structure logging for a distributed system?

Structured JSON to stdout, one event per line, shipped by the platform. Boot 3.4+ has built-in structured logging — no more hand-rolled Logback encoders:

```
logging:
  structured:
    format.console: ecs          # or logstash, gelf
    level.com.acme: INFO
```

Principles: never log secrets, tokens or PII (and know your obligations — GDPR makes logs a data store you must be able to purge); include `traceId` and `spanId` in every line; log at boundaries and on decisions, not inside loops; use levels meaningfully — `ERROR` means a human should look, and an `ERROR` that fires routinely trains everyone to ignore the channel.

The senior point: logs are the **most expensive** telemetry per unit of insight. A team that logs every request body at INFO will spend more on log storage than on compute, and still struggle to find anything. Sample high-volume logs, and prefer a metric for anything you would otherwise count by grepping.

Q65. What is your production readiness checklist for a new service?

Grouped, so it reads as judgment rather than recitation:

Operability — health probes split into liveness and readiness, graceful shutdown, resource requests and limits, a dashboard, and alerts tied to user-visible symptoms rather than causes.

Reliability — timeouts on every outbound call, retries only where idempotent, circuit breakers on critical dependencies, and a documented, *tested* rollback path. Untested rollback is not a rollback.

Observability — RED metrics per endpoint, tracing wired end to end, structured logs with trace ids, and an SLO with an error budget so "is this bad enough to page someone" has an agreed answer.

Security — no secrets in images, actuator on a management port, dependency scanning in CI, TLS internally, and least-privilege database credentials.

Data — migrations tested against a production-like database, backups *and a restore that has actually been performed*, and a retention policy.

Process — a runbook covering the top three failure modes, an on-call owner, and load testing that established where the service breaks before users find out.

The single strongest thing to say: "*and we test the failure paths*" — chaos drills, dependency failure injection, restore rehearsals. Everyone has a checklist; few have evidence it works.

11. Spring Boot 4 — What Changed and Why

As of September 2026, Spring Boot 4.1 is the current line (4.1.0 released 30 June 2026, 4.1.1 in August). Spring Boot 3.5 reached the end of open-source support on 30 June 2026. Expect at least one question on this — it is the cheapest way for an interviewer to find out whether you keep current.

Q66. What is new in Spring Boot 4 / Spring Framework 7?

Six headline changes:

Modularization. The codebase was split into 70+ focused jars named `spring-boot-
<technology>`, with root packages `org.springframework.boot.<technology>`. Smaller classpath, fewer conditions to evaluate, better AOT and native-image behaviour.

Null safety with JSpecify. Packages are `@NotNullMarked`, meaning types are non-null by default and `@Nullable` marks the exceptions. With a compatible checker this moves a class of NPEs from runtime to compile time, and it applies across the whole Spring portfolio.

API versioning, as covered in Q23 — `version = "1.0"` on mappings, with resolution by header, path segment, query parameter or media type, plus deprecation headers.

HTTP service clients — `@ImportHttpServices` registers `@HttpExchange` interfaces as beans declaratively, removing the proxy-factory boilerplate (Q26).

Built-in resilience — `@Retryable` with exponential backoff and jitter, and `@ConcurrencyLimit`, in core Spring rather than requiring Resilience4j for simple cases (Q49).

Java 25 support with Java 17 retained as the baseline — notably *not* a forced upgrade, and the Jakarta EE 11 / Servlet 6.1 move needs no `javax` → `jakarta` rewrite. This is a much gentler major version than 2.7 → 3.0 was.

Spring Boot **4.1** then added Spring gRPC, lazy JDBC connection acquisition, an `InetAddressFilter` for SSRF protection, `@RedisListener` for Redis messaging, and compiler-checked type-safe property paths in Spring Data.

Q67. What actually breaks when you upgrade from Boot 3 to Boot 4?

This is the more useful question, and the answer is mostly "dependencies and names":

Starter renames. `spring-boot-starter-web` → `spring-boot-starter-webmvc`; `spring-boot-starter-web-services` → `spring-boot-starter-webservices`; `spring-boot-starter-aop` → `spring-boot-starter-aspectj`; OAuth2 starters gained a `security-` prefix, e.g. `spring-boot-starter-security-oauth2-client`.

Package moves. `BootstrapRegistry` moved to `org.springframework.boot.bootstrap`, `EnvironmentPostProcessor` to `org.springframework.boot`, `@EntityScan` to `org.springframework.boot.persistence.autoconfigure`.

Jackson 3. Group ids move from `com.fasterxml.jackson` to `tools.jackson`; `ObjectMapper` gives way to `JsonMapper`; `Jackson2ObjectMapperBuilderCustomizer` → `JsonMapperBuilderCustomizer`; `@JsonComponent` → `@JacksonComponent`; `@JsonMixin` → `@JacksonMixin`; properties reorganised under `spring.jackson.json.read` / `.write`.

Testing. `@MockBean` / `@SpyBean` removed in favour of `@MockitoBean` / `@MockitoSpyBean`; `@SpringBootTest` no longer auto-configures `MockMvc`, `WebClient` or `TestRestTemplate`.

Removals. Undertow (Servlet 6.1 incompatibility), Pulsar Reactive auto-configuration, Spring Session Hazelcast and MongoDB, Spock integration, embedded launch scripts, the classic uber-jar loader.

Defaults and properties. DevTools Live Reload off by default; liveness/readiness probes on by default; MongoDB properties renamed (`spring.data.mongodb` → `spring.mongodb`); Spring Session properties reorganised under `spring.session.data.*`.

The practical migration advice — worth stating because it shows you have done upgrades: go to the **latest 3.5.x first**, fix every deprecation warning there, migrate `@MockBean` to `@MockitoBean` (available since 3.4), then take the major step. Use OpenRewrite recipes for the mechanical renames, and expect the first day's pain to be missing transitive dependencies rather than code changes.

Q68. Should a team upgrade to Boot 4 now?

A good senior answer weighs it rather than cheerleading.

Reasons to move: Boot 3.5 lost open-source support in June 2026, so you are on commercial support or unpatched; 4.x is where new features and the ecosystem's attention now are; and the upgrade is substantially less disruptive than 3.0 was because there is no Jakarta namespace migration and Java 17 still works.

Reasons to wait: the modularization means third-party starters and internal shared libraries need their own updates first, and a large estate upgrades at the speed of its slowest dependency. If you maintain internal starters, they are the critical path — do those first.

How I would sequence it: upgrade the shared libraries and internal starters, pick one low-risk service as a pilot to discover the estate-specific surprises, write the findings into a migration guide, then roll out with OpenRewrite doing the mechanical work. Keep security patching on the old line until the majority has moved.

Being able to say "here is how I would sequence it across twenty services" is what the question is actually testing at a senior level — not whether you memorised the release notes.

12. Rapid-Fire Round

Short questions used as warm-ups or filler between deeper topics. Answer in two or three sentences and stop — over-explaining a simple question reads as nervousness.

What does `@SpringBootApplication` combine? `@SpringBootApplication`, `@EnableAutoConfiguration` and `@ComponentScan`. Placed in the root package so scanning covers everything below it.

What is a starter? A dependency aggregator — a pom with no code that pulls in a coherent, version-aligned set of libraries plus the matching auto-configuration.

How does an executable jar work? A nested jar with Boot's own `JarLauncher` as `Main-Class`; it installs a custom class loader that reads nested `BOOT-INF/lib/*.jar` entries without unpacking. Boot 4 removed the classic loader in favour of the newer implementation.

`@RestController` vs `@Controller`? `@RestController` = `@Controller` + `@ResponseBody`; return values are serialized to the response body rather than resolved as view names.

`@RequestParam` vs `@PathVariable`? Query string versus URI template segment. Path for identifying a resource, query for filtering, sorting and pagination.

`@RequestBody` vs `@ModelAttribute`? JSON/XML deserialization via `HttpMessageConverter` versus binding form/query parameters onto an object.

What is `CommandLineRunner` / `ApplicationRunner`? Beans run once after the context is ready — the former gets raw `String[]` args, the latter parsed `ApplicationArguments`. For startup tasks; order with `@Order`.

Difference between `@Component`, `@Service`, `@Repository`? Functionally interchangeable stereotypes, except `@Repository` adds exception translation from vendor `SQLExceptions` to Spring's `DataAccessException` hierarchy. The rest is documentation of intent.

What is `DispatcherServlet`? The front controller: resolves a handler via `HandlerMapping`, invokes it through a `HandlerAdapter`, resolves the return value, and handles exceptions via `HandlerExceptionResolver`.

`@Async` — what do you need for it to work? `@EnableAsync`, a call through the proxy (not self-invocation), a `void` or `Future/CompletableFuture` return type, and an explicitly configured executor — the default `SimpleAsyncTaskExecutor` creates an unbounded number of threads, which is a production hazard unless virtual threads are enabled.

How do you schedule a job? `@EnableScheduling` plus `@Scheduled(cron = ...)`. In a multi-instance deployment it runs on **every** instance — use `ShedLock` or a distributed scheduler if it must run once.

What is `spring.factories` used for now? Largely superseded. Auto-configuration moved to `AutoConfiguration.imports` in 2.7 (the old key was removed in 3.0); `spring.factories` still registers things like `FailureAnalyzer`, `EnvironmentPostProcessor` and `ApplicationListener`.

Actuator endpoints you would expose in production? `health` (with probe groups), `info`, `prometheus`, and `metrics` — on a separate management port, secured. Never `env`, `heapdump`, `threaddump` or `shutdown` on a public interface.

@ControllerAdvice vs @RestControllerAdvice? The latter adds `@ResponseBody`. Both can be scoped to specific packages, annotations or controller types.

What is `HandlerInterceptor` vs a servlet `Filter`? Filters sit outside Spring MVC and see every request including static resources; interceptors are MVC-aware and know the handler being invoked, which makes them the right place for handler-specific concerns.

How do you handle file uploads? `MultipartFile` with `spring.servlet.multipart.max-file-size` and `max-request-size`. For large files, stream rather than buffering — and validate content type by inspecting the bytes, not by trusting the extension or the client-supplied type.

What is `@Modifying` for? Marks a `@Query` as an update or delete. Pair with `clearAutomatically = true` / `flushAutomatically = true` when the persistence context would otherwise hold stale entities — a common source of "my update didn't take" confusion.

Difference between `save` and `saveAndFlush`? `save` may only queue the change until the transaction flushes; `saveAndFlush` forces the SQL immediately — useful when you need a generated value or a constraint violation to surface before the method ends.

`getReferenceById` vs `findById`? The former returns a lazy proxy without hitting the database (throwing on access if the row is absent), and is ideal for setting a foreign-key association without loading the whole entity; the latter fetches and returns an `Optional`.

What is the difference between `@Bean` `initMethod` and `@PostConstruct`? Both run at initialization; `@PostConstruct` is the JSR-250 standard and runs slightly earlier, while `initMethod` is used for third-party classes you cannot annotate.

How do you expose build info in `/actuator/info`? Add the `build-info` goal to the Spring Boot Maven/Gradle plugin; it generates `META-INF/build-info.properties`, which the `InfoContributor` picks up. Git details come from `git-commit-id-plugin`.

Closing question to ask them

Interviews run both ways, and a good closing question does more for you than a perfect answer to any of the above. Some that consistently produce useful signal:

"What does the deployment pipeline look like from merge to production, and how long does it take?" — tells you about automation maturity and how much of the day is spent waiting.

"What is the oldest piece of the system, and what would it take to replace it?" — tells you whether technical debt is discussed honestly.

"When something breaks at 2am, what happens?" — tells you about on-call load, observability, and blame culture in one question.

"What did the last postmortem change?" — the difference between a team that writes documents and a team that learns.